

AI Development Environment

Setup, Startup, Docker & Deployment Workflow

Andrew Mitchell — The Aging Apprentice

GitHub: theAgingApprentice

Table of Contents

1. Environment Overview

- 1.1 Hardware
- 1.2 Installed Software
- 1.3 GitHub Credentials
- 1.4 Architecture Summary

2. Daily Startup Checklist

- 2.1 Step 1 — Verify Ollama is Running
- 2.2 Step 2 — Start Docker Desktop
- 2.3 Step 3 — Open VS Code and Your Project
- 2.4 Step 4 — Pull Latest Code from GitHub
- 2.5 Step 5 — Start Backend and Frontend via Docker Compose
- 2.6 Step 6 — Verify End-to-End
- 2.7 Full Startup at a Glance

3. Docker in the AI-First Stack

- 3.1 What to Containerise
- 3.2 Why Containerise the Backend
- 3.3 Connecting Containers to Native Ollama
- 3.4 Recommended Project Structure with Docker
- 3.5 Minimal Backend Dockerfile
- 3.6 Useful Docker Commands

4. GitHub Workflow

- 4.1 Daily Commit Routine
- 4.2 Recommended .gitignore

5. Migrating to a Local Server

- 5.1 Target Architecture on the Server
- 5.2 What Changes Between Mac and Server
- 5.3 One-Time Server Setup
- 5.4 Deploying an App to the Server

5.5 Server docker-compose.yml Differences

6. Mac Mini as Your AI Server

- 6.1 Why Mac Mini Works So Well for AI
- 6.2 The Right Spec for Gemma 4
- 6.3 Single Mac Mini vs. Cluster
- 6.4 Clustering with Exo
- 6.5 Mac Mini vs. Linux + NVIDIA GPU
- 6.6 Recommended Migration Path

7. CI/CD with GitHub Actions

- 7.1 What GitHub Actions Can Do
- 7.2 What GitHub Actions Cannot Do
- 7.3 Recommended Approach: Build in the Cloud, Deploy Locally
- 7.4 Optional: Automated Polling on the Server
- 7.5 Full Pipeline Summary

8. App Planning & Development Process

- 8.1 Phase 1 — Define the Conversation
- 8.2 Phase 2 — Design the Database Schema
- 8.3 Phase 3 — Define Tool Functions
- 8.4 Phase 4 — Build and Iterate
- 8.5 Phase 5 — Testing
- 8.6 Phase 6 — System Prompt Tuning

9. Quick Reference

This table of contents is a static listing. Update section numbers here whenever sections are added or reordered.

1. Environment Overview

This document describes the local AI development environment running on the Mac Studio, how to start it each session, how Docker fits into the stack, and how to migrate and deploy apps to a local server — including using a Mac Mini as a dedicated inference server.

1.1 Hardware

Machine	Apple Mac Studio
Chip	Apple M4 Max

Memory	36 GB unified memory
OS	macOS 26.3.1

1.2 Installed Software

Ollama	Local LLM inference server — runs Gemma 4 natively via Metal
Model	gemma4:26b (Q4_K_M, 18 GB, 256K context window)
Docker Desktop	Container runtime for backend services and deployment parity
VS Code	Primary IDE for all development
Git / GitHub	Version control — username: theAgingApprentice
GitHub PAT	Stored securely in macOS Keychain via osxkeychain credential helper
Python 3	Backend development (FastAPI + uvicorn)
Node.js / npm	JavaScript tooling and frontend builds

1.3 GitHub Credentials

Git is configured globally at `~/.gitconfig` with username `theAgingApprentice`. Authentication credentials (personal access token) are managed by the macOS `osxkeychain` credential helper and stored securely in the system keychain — never in plain text. No manual credential entry is required after initial setup.

1.4 Architecture Summary

The AI stack keeps Ollama running natively on macOS for full Metal and unified memory performance. All other services run in Docker containers during development, mirroring the production server layout and eliminating environment drift.

- Browser frontend — conversational chat UI, no traditional forms
- FastAPI backend (Docker) — manages context, tool dispatch, streaming
- Ollama — native macOS process, accessed by containers via `host.docker.internal`
- SQLite databases — mounted as Docker volumes, persisted on the host filesystem
- GitHub — all source code, CI pipeline definitions, release tags

2. Daily Startup Checklist

Follow these steps at the start of every development session. The full sequence takes under two minutes once the environment is established.

2.1 Step 1 — Verify Ollama is Running

Ollama must be running before any backend container can reach Gemma 4. Check the menu bar for the Ollama icon. If present, the server is already running. If not, open Terminal and run:

```
ollama serve
```

Confirm the API is responding:

```
curl http://localhost:11434/api/tags
```

Expected response includes gemma4:26b in the models list. If connection-refused, re-run ollama serve and wait 10 seconds before retrying.

2.2 Step 2 — Start Docker Desktop

Open Docker Desktop from Applications or the menu bar. Wait for the whale icon to stop animating. Verify from Terminal:

```
docker info | head -5
```

Enable 'Start Docker Desktop when you log in' in Preferences → General to make this automatic.

2.3 Step 3 — Open VS Code and Your Project

1. Launch VS Code.
2. Open your project: File → Open Folder, or drag the folder onto the VS Code icon.
3. Open the integrated terminal: Ctrl+` (backtick).
4. Confirm directory: pwd

2.4 Step 4 — Pull Latest Code from GitHub

```
git pull origin main
```

If you have uncommitted local changes, run git stash first, then pull, then git stash pop.

2.5 Step 5 — Start Backend and Frontend via Docker Compose

From your project root:

```
docker compose up --build    # first run or after dependency changes
docker compose up           # subsequent starts
```

Confirm the backend API is live:

```
open http://localhost:8000/docs
```

| *Run in the background with: `docker compose up -d`. View logs with: `docker compose logs -f`*

2.6 Step 6 — Verify End-to-End

```
curl -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "Hello, are you ready?"}'
```

A valid JSON response with an assistant message confirms everything is working.

2.7 Full Startup at a Glance

```
# Terminal 1 — Ollama (if not auto-started)
ollama serve &

# Terminal 2 — Project
cd ~/projects/my-app
git pull origin main
docker compose up
```

3. Docker in the AI-First Stack

Docker plays a different role in AI-first apps than in traditional web apps. Ollama must run natively on macOS to access Metal GPU acceleration and unified memory. Everything else benefits from containerisation.

3.1 What to Containerise

FastAPI backend	Full container — Python, dependencies, environment variables
SQLite database	Data mounted as a named Docker volume, not inside the container
Frontend (React)	Full container with Node build stage + nginx to serve static files
Ollama / Gemma 4	Native macOS only — never containerise on Mac (loses Metal / GPU)

3.2 Why Containerise the Backend

- Dependency isolation — Python version and packages are locked, no conflicts between projects
- Environment parity — same container image runs on Mac, the local server, or any future host
- Clean teardown — docker compose down removes all processes without touching code or data
- Easier onboarding — anyone cloning the repo gets a working environment with one command

3.3 Connecting Containers to Native Ollama

Containers use `host.docker.internal` to reach services running on the Mac host. Set this in `docker-compose.yml`:

```
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    volumes:
      - db_data:/app/data
    environment:
      - OLLAMA_URL=http://host.docker.internal:11434
      - MODEL=gemma4:26b
  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
volumes:
  db_data:
```

The db_data named volume persists your SQLite database between container restarts. Only docker compose down -v deletes volumes.

3.4 Recommended Project Structure with Docker

```
my-app/
├── backend/
│   ├── Dockerfile
│   ├── main.py          # FastAPI entry point
│   ├── tools.py         # Gemma 4 tool definitions
│   ├── database.py      # SQLite helpers
│   └── requirements.txt
├── frontend/
│   ├── Dockerfile
│   └── index.html
├── data/                # host-side DB files (gitignored)
├── docker-compose.yml
├── .env                 # local secrets (gitignored)
├── .gitignore
└── README.md
```

3.5 Minimal Backend Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
```

3.6 Useful Docker Commands

docker compose up	Start all services (add -d to detach)
docker compose up --build	Rebuild images before starting
docker compose down	Stop containers (volumes preserved)
docker compose logs -f	Follow live logs from all services
docker compose ps	List running containers and status
docker compose exec backend bash	Open a shell inside the backend container
docker volume ls	List all named volumes
docker system prune	Remove stopped containers and unused images

4. GitHub Workflow

Each app lives in its own GitHub repository under theAgingApprentice. Authentication is handled transparently by the osxkeychain credential helper. Follow this workflow to keep history clean and main branch stable.

4.1 Daily Commit Routine

1. Start with git pull origin main.
2. Do your work. Save files frequently.
3. Stage changes: git add . or use VS Code Source Control panel (Ctrl+Shift+G).
4. Commit: git commit -m "Add recipe search tool"
5. Push: git push origin main

4.2 Recommended .gitignore

```
.venv/  
__pycache__/  
*.pyc  
.env  
data/  
.DS_Store  
node_modules/
```

Never commit .env files. Store secrets locally and inject them at runtime via docker-compose.yml or the server environment.

5. Migrating to a Local Server

When an app is stable on the Mac Studio, the next step is deploying it to a dedicated local server. This server runs Linux (Ubuntu recommended), hosts Docker, and runs Ollama natively. The server is private — it is not accessible from the internet or from external services like GitHub.

5.1 Target Architecture on the Server

OS	Ubuntu 22.04 LTS or 24.04
Ollama	Native Linux install — with NVIDIA GPU support if available

Model	gemma4:26b — pulled once with: ollama pull gemma4:26b
Docker	Docker Engine + Docker Compose plugin (not Docker Desktop)
Networking	LAN only — no internet exposure, accessible from Mac via local IP
Reverse proxy	Nginx or Caddy — routes traffic to backend and frontend containers

5.2 What Changes Between Mac and Server

- OLLAMA_URL changes from host.docker.internal:11434 to the Docker bridge IP (172.17.0.1:11434)
- The --reload flag is removed from uvicorn; use a fixed number of workers instead
- docker-compose.yml gains restart: unless-stopped so services survive reboots
- A reverse proxy (Nginx) sits in front of the backend
- Environment variables are managed via a .env file on the server, not committed to GitHub

5.3 One-Time Server Setup

1. Install Ubuntu and connect to your LAN. Note the static IP (e.g. 192.168.1.50).
2. Install Docker Engine: follow docs.docker.com/engine/install/ubuntu
3. Install Ollama: curl -fsSL https://ollama.com/install.sh | sh
4. Pull the model: ollama pull gemma4:26b
5. Enable Ollama on boot: sudo systemctl enable ollama
6. Create a deploy user and add to the docker group.
7. Create the app directory: mkdir -p /opt/apps/recipe-app

5.4 Deploying an App to the Server

Because the server has no inbound internet access, GitHub cannot push to it directly. Instead you pull from the Mac using SSH:

1. Push your release to GitHub: git push origin main
2. SSH into the server: ssh deploy@192.168.1.50
3. On the server, pull the latest code:

```
cd /opt/apps/recipe-app && git pull origin main
```

4. Rebuild and restart containers:

```
docker compose pull && docker compose up --build -d
```

5. Verify: docker compose ps

*Create a `deploy.sh` script on the server with these commands. Then deploy in one line from the Mac:
`ssh deploy@192.168.1.50 'opt/apps/recipe-app/deploy.sh'`*

5.5 Server docker-compose.yml Differences

```
services:
  backend:
    build: ./backend
    restart: unless-stopped
    environment:
      - OLLAMA_URL=http://172.17.0.1:11434 # Docker bridge to host
      - MODEL=gemma4:26b
    command: uvicorn main:app --host 0.0.0.0 --port 8000 --workers 2
```

172.17.0.1 is the Linux host as seen from inside a container. Verify on your server with: `ip route | grep docker`. This replaces `host.docker.internal` which is macOS-specific.

6. Mac Mini as Your AI Server

A Mac Mini running Apple Silicon is one of the best choices for a private local AI inference server. The same unified memory architecture that makes the Mac Studio excellent for AI development makes a Mac Mini M4 Pro an ideal always-on server — with a fraction of the power consumption and noise of any comparable NVIDIA-based machine.

6.1 Why Mac Mini Works So Well for AI

The fundamental advantage is unified memory. On a traditional PC, model weights must be copied from system RAM to GPU VRAM across the PCIe bus before inference begins. On Apple Silicon, the CPU, GPU, and Neural Engine all share one memory pool — a 48 GB Mac Mini M4 Pro can feed the entire Gemma 4 26B model to the GPU cores instantly, with no transfer overhead.

Memory bandwidth	M4 Pro: 273 GB/s — double the base M4 and critical for fast token generation
Power draw	30–40 W under full AI inference load vs. 350–450 W for an RTX 4090 system
Electricity cost	Approximately \$52 CAD/year running 24/7 at 40 W
Noise	Near-silent under load — no active cooling required
Form factor	5 × 5 inches — fits on a shelf or mounts behind a monitor

OS advantage	Identical macOS stack to your Mac Studio — same Ollama, same Docker commands
---------------------	--

6.2 The Right Spec for Gemma 4

Memory is the critical variable. The 26B model requires 16–18 GB at 4-bit quantisation, but you must also account for the OS, Docker overhead, and context window growth during long conversations.

Mac Mini M4 (base, 16 GB)	Too tight for Gemma 4 26B. Suitable for 7B–8B models only.
Mac Mini M4 Pro, 24 GB	Workable but leaves little headroom. Better for 14B models.
Mac Mini M4 Pro, 48 GB	Sweet spot for Gemma 4 26B. Comfortable inference with OS overhead.
Mac Mini M4 Pro, 64 GB	Headroom for 31B model or running two apps simultaneously.

Avoid the base M4 chip for this workload. The M4 Pro has 273 GB/s memory bandwidth vs. 120 GB/s on the base M4. That bandwidth difference directly determines tokens-per-second and whether responses feel instant or sluggish.

The M4 Pro 48 GB Mac Mini costs approximately \$2,000–\$2,200 CDN and is the recommended single-node server for running Gemma 4 26B. The 64 GB model at ~\$2,400–\$2,600 CDN is worth it if you plan to run the 31B model or serve multiple apps.

6.3 Single Mac Mini vs. Cluster

A single well-specced Mac Mini handles all current use cases for the recipe and fitness apps comfortably. Clustering becomes relevant when you want to run 70B+ models or serve many concurrent users.

Single M4 Pro 48 GB	Runs Gemma 4 26B at ~30–40 tok/s. Handles personal apps and dev workloads.
Single M4 Pro 64 GB	Runs Gemma 4 31B. Good headroom for future larger models.
2-node cluster (96–128 GB)	Runs 70B models split across nodes using exo framework.
4-node cluster (192+ GB)	Enterprise-scale inference. Overkill for personal apps but viable for small teams.

6.4 Clustering with Exo

Exo (github.com/exo-explore/exo) is the open-source framework for distributing a single LLM's inference across multiple Apple Silicon nodes. Each node handles a portion of the model's layers, and results are assembled before returning a response.

- Requires M4 Pro or better — the M4 Pro's Thunderbolt 5 supports RDMA which reduces inter-node latency from ~300 μ s to under 50 μ s
- The base M4 uses Thunderbolt 4 and works with exo, but is slower on the network side
- Connect nodes via Thunderbolt 5 in a daisy-chain or ring topology
- Ethernet (2.5 Gbps+) works as a fallback if Thunderbolt cables are unavailable

Exo currently uses mDNS for node discovery and does not fully utilise the Thunderbolt bridge for inference traffic — it defaults to whichever IP it finds first. The Thunderbolt mesh is most useful for fast file transfers between nodes rather than inference routing. This is expected to improve with future exo releases.

6.5 Mac Mini vs. Linux + NVIDIA GPU

	Mac Mini M4 Pro
Cost (inference-ready)	\$2,000–\$2,400 CDN
Power (inference)	30–40 W
Noise	Near-silent
Setup friction	Zero — same macOS stack
Max VRAM	64 GB unified (no limit gap)
Token speed (26B)	~30–40 tok/s
Training workloads	Light PEFT only
Ollama support	Native Metal, day-one

For inference-only workloads on a private LAN — which is exactly your use case — the Mac Mini wins on every practical factor except raw token throughput. If speed becomes a bottleneck later, a GPU server can be added alongside the Mac Mini rather than replacing it.

6.6 Recommended Migration Path

Stage 1 (now)	Develop and test on Mac Studio M4 Max. Run Ollama + Docker locally.
Stage 2 (first server)	Mac Mini M4 Pro 48 GB (~\$2,000 CDN). Always-on LAN inference server. Same Ollama + Docker stack, zero new tools to learn.

Stage 3 (scale up)	Add a second Mac Mini and run <code>exo</code> to pool memory for 70B+ models, or upgrade to 64 GB for 31B inference.
Stage 4 (optional)	Add a Linux + NVIDIA GPU node for high-throughput batch workloads or model fine-tuning, while keeping Mac Minis for interactive inference.

When you provision the Mac Mini server, the only changes to your existing workflow are: (1) update `OLLAMA_URL` in `docker-compose.yml` to `172.17.0.1:11434`, (2) remove `--reload` from the `uvicorn` command, (3) add `restart: unless-stopped` to all services. Everything else — `git pull`, `docker compose up`, `deploy.sh` — is identical.

7. CI/CD with GitHub Actions

GitHub Actions automates testing and build steps in the cloud. Because your local server has no inbound internet access, GitHub Actions cannot deploy directly to it. The solution is a pull-based deployment model: GitHub Actions validates and packages the code, and the server pulls the result on demand.

7.1 What GitHub Actions Can Do

- Run automated tests on every push (pytest for Python, Jest for JavaScript)
- Lint and type-check code before it reaches the main branch
- Build Docker images and push them to GitHub Container Registry (ghcr.io)
- Tag releases and generate changelogs automatically
- Notify you when a build passes or fails

7.2 What GitHub Actions Cannot Do (in Your Setup)

- Push code or images directly to your local server — the server is not internet-accessible
- Run deployment commands on the server via SSH from GitHub — same reason

Never expose your local server to the internet to work around this limitation. The pull-based model is the correct and secure approach for a private LAN server.

7.3 Recommended Approach: Build in the Cloud, Deploy Locally

GitHub Actions handles the build and test stage. You (or a cron job on the server) handle deployment by pulling from GitHub when ready.

Stage 1 — CI pipeline (.github/workflows/ci.yml)

```
name: CI
on:
  push:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install -r backend/requirements.txt
      - run: pytest backend/tests/
  build-image:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: theAgingApprentice
          password: ${ secrets.GITHUB_TOKEN }
      - uses: docker/build-push-action@v5
        with:
          context: ./backend
          push: true
          tags: ghcr.io/theagingapprentice/recipe-app:latest
```

Stage 2 — Server deploy.sh

```
#!/bin/bash
echo $CR_PAT | docker login ghcr.io -u theAgingApprentice --password-stdin
cd /opt/apps/recipe-app
git pull origin main
docker compose pull
docker compose up -d
docker image prune -f
```

Run from the Mac via SSH:

```
ssh deploy@192.168.1.50 '/opt/apps/recipe-app/deploy.sh'
```

CR_PAT is a GitHub Personal Access Token with read:packages scope, stored in the server's shell profile. It requires only read access to the container registry, separate from your development PAT.

7.4 Optional: Automated Polling on the Server

```
# crontab -e on the server
*/10 * * * * /opt/apps/recipe-app/deploy.sh >> /var/log/deploy.log 2>&1
```

Automated polling deploys automatically when CI passes. Only enable this once your tests are thorough enough that a passing build is genuinely safe to deploy.

7.5 Full Pipeline Summary

1. Dev on Mac	Write code, run locally via docker compose up
2. Push to GitHub	git push origin main — triggers GitHub Actions
3. CI runs	Tests pass, image built and pushed to ghcr.io
4. Deploy	SSH to server and run deploy.sh, or wait for cron poll
5. Server pulls	docker compose pull + up -d — new version live on LAN

8. App Planning & Development Process

Follow this structured process for every new app or major feature. AI-first apps require extra design thought upfront because the model's behaviour depends heavily on how tools and prompts are shaped.

8.1 Phase 1 — Define the Conversation

1. Write 10–20 example user messages (e.g. show me pasta dishes under 30 minutes).
2. For each, define the expected behaviour: DB query, clarifying question, or direct answer.
3. Identify which requests need a database lookup vs. model knowledge alone.
4. Document edge cases: ambiguous requests, out-of-scope questions, empty results.

8.2 Phase 2 — Design the Database Schema

1. List all entities (e.g. recipes, ingredients, tags, workouts, exercises).
2. Define relationships and junction tables (e.g. recipe_ingredients).

3. Identify fields that will be searched or filtered — these need indexes.
4. Create the schema and seed with 10–20 test records before building tools.

8.3 Phase 3 — Define Tool Functions

Tool functions bridge Gemma 4 and the database. Define name, description, and parameters before implementing. The description is what the model reads to decide when to call the tool.

- Name: short and descriptive (e.g. `search_recipes`, `log_workout`)
- Description: one sentence explaining when and why to call this tool
- Parameters: clearly typed (e.g. `query: str`, `max_results: int = 5`)
- Return: consistent JSON the backend parses and presents to the user

Test each tool by calling it directly in Python before connecting it to the LLM. Confirm correct results for typical, edge, and empty-result cases.

8.4 Phase 4 — Build and Iterate

1. Implement FastAPI with one `/chat` endpoint and one tool to start.
2. Get end-to-end flow working (browser → backend → Ollama → tool → response) before adding features.
3. Add one tool at a time. Commit to GitHub after each working addition.
4. Only add UI complexity after the conversation layer is solid.

8.5 Phase 5 — Testing

Database and tool testing

- Unit test every tool with at least three cases: typical, edge, and empty result.
- Test SQL queries in DB Browser for SQLite before embedding in code.
- Confirm the API returns correct JSON using Swagger UI at `/docs`.

LLM behaviour testing

- Run all example conversations from Phase 1. Verify the model calls the correct tool each time.
- Test ambiguous inputs — does the model ask for clarification or guess?
- Test out-of-scope inputs — does the model handle them without hallucinating?

Regression testing

- After any change to a tool or system prompt, re-run all example conversations.
- Keep a passing test log in the repository at `tests/conversations.md`.

8.6 Phase 6 — System Prompt Tuning

- Start minimal. Add constraints only when you observe problems.
- Explicitly instruct the model to use only tool results and never invent data.
- Specify tone and format (e.g. respond in 1–3 sentences, then show results).
- Test the same inputs before and after any system prompt change.

9. Quick Reference

Ollama API (Mac)	http://localhost:11434
Ollama API (container)	http://host.docker.internal:11434
Ollama API (Linux server)	http://172.17.0.1:11434
FastAPI backend	http://localhost:8000
FastAPI Swagger docs	http://localhost:8000/docs
Frontend (Vite)	http://localhost:5173
GitHub	github.com/theAgingApprentice
Container registry	ghcr.io/theagingapprentice
Model	gemma4:26b (Q4_K_M, 18 GB, 256K context)
Recommended server HW	Mac Mini M4 Pro 48 GB — ~\$2,000 CDN
Server LAN IP	192.168.1.50 (update when provisioned)
Server deploy path	/opt/apps/<app-name>

Update the server LAN IP and deploy path once the server is provisioned. Keep this document current as the stack evolves.